



UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
ESCUELA DE COMPUTACIÓN
CAMPUS BÁRBULA
ARQUITECTURA DEL COMPUTADOR



Implementacion de Recursividad en MIPS32

Bárbula, mayo de 2026

Daniel Gutierrez CI: 32047059
Gabriel Diaz CI: 31161550

Introducción

MIPS32 un lenguaje de programación que se comunica directamente al procesador, un lenguaje de bajo nivel, donde la eficiencia está por encima de todo aspecto del código.

En el presente informe se estudiará la pila, cómo funciona y los problemas recursivos y qué importancia tiene el profundo manejo de ésta. A su vez, podremos analizar qué pasa con la eficiencia, el procesador; cuando hacemos iterativo un algoritmo recursivo.

Aquí es donde se plantea la pregunta ¿Qué algoritmo es más rápido? uno donde no tocamos la pila u otro donde tenemos que profundizar en el manejo de ella.

La arquitectura MIPS32 representa una inmersión directa en la realidad del hardware, donde la abstracción de los lenguajes de alto nivel desaparece por completo. En este entorno, el control de la memoria, el uso eficiente de los registros y la gestión del flujo de ejecución recaen exclusivamente en las decisiones del programador, convirtiendo a la eficiencia en el pilar fundamental del diseño de software.

El presente informe aborda el estudio práctico y conceptual de la recursividad en MIPS32 y su interacción directa con la memoria RAM a través de la pila (*stack*). A diferencia de un ciclo iterativo simple, la recursividad en ensamblador exige la construcción manual de marcos de apilación (*stack frames*) mediante el puntero de pila (**\$sp**) para preservar el contexto de ejecución (como el registro de retorno **\$ra** y los argumentos **\$a0**).

A lo largo de este documento se analiza la transición de un algoritmo recursivo a uno iterativo, evaluando los costos en tiempo de procesamiento y consumo de memoria. Asimismo, se discuten los riesgos de desbordamiento (*Stack Overflow*), las divergencias entre los modelos teóricos de los libros y la implementación real en el simulador MARS, y la justificación técnica de cuándo la recursividad es una necesidad estructural frente al enfoque iterativo.

Implementación de la recursividad en MIPS32

En MIPS32 la recursividad se implementa haciendo que una etiqueta se "llame" a sí misma usando la instrucción `jal`.

Sin embargo, esto genera el siguiente problema: ¿Qué pasa si se repite el mismo bloque de instrucciones? Por supuesto, se repetirá el proceso, por ende se repetirán instrucciones las cuales pueden borrar datos importantes que se desean preservar entre "llamadas", es por eso que se utiliza la pila de la siguiente forma:

Se debe de identificar cuantos elementos tienes que salvar inicialmente, por ejemplo si debes guardar 4 elementos, debes abrir 4*4 espacios en la pila, para abrirlos se restan la cantidad de espacios a abrir.

Se utiliza la instrucción `sw` para guardar uno a uno los elementos (en el mayor de los casos son los parámetros) y cuando se termine la llamada, se deben de sacar uno a uno utilizando la instrucción `lw` y regresando la pila (sumando la cantidad de espacios).

Esto es como la base y lo primero que se debe de dominar para realizar correctamente la recursividad en MIPS32.

En resumen: la pila actúa como un bloc de notas temporal en la memoria RAM. Antes de que la función se vuelva a llamar a sí misma, "guarda" (usando la instrucción `sw`) los valores importantes en la pila. Cuando la recursividad termina y toca devolverse, la función "saca" (usando `lw`) esos datos de la pila en orden inverso para poder regresar al punto exacto donde se quedó sin perder información.

¿Qué riesgos de desbordamiento existen?

El riesgo principal es el temido **Stack Overflow (desbordamiento de pila)**. Como la memoria RAM de la pila tiene un límite físico, si una función recursiva se llama a sí misma demasiadas veces, la pila crece tanto que termina invadiendo o chocando con otras áreas de la memoria, haciendo que el programa colapse.

¿Cómo lo mitigamos?

- **Asegurar el caso base:** Es vital programar correctamente la condición de parada (por ejemplo, cuando el número llega a 0) para garantizar que la recursividad termine en algún momento.
- **Limitar la entrada:** Si sabemos que el programa recibirá un número enorme que va a agotar la memoria, es mejor abandonar la recursividad y usar un enfoque iterativo normal (un ciclo)

Aun así, problemas recursivos muy grandes podrían abarcar muchas mas llamadas de las que puede aguantar la pila, por esto el verdadero limite lo asigna el sistema operativo, el lenguaje (en este caso MIPS32) o el simulador utilizado.

MIPS32 utiliza un sistema de direccionamiento de 32 bits, lo que le permite mapear un espacio total de 4 GB de memoria virtual.

- En los sistemas operativos estándar (como Linux para MIPS), este espacio se divide a la mitad de forma estricta.
- El espacio de usuario (**kuseg**) obtiene los 2 GB inferiores (direcciones `0x00000000` a `0x7FFFFFFF`).

- Como la pila de llamadas (*stack*) crece hacia abajo desde el límite superior del espacio de usuario (comenzando típicamente en `0x7FFF_FFFF` o `0x7FFF_FFF0`), su tamaño máximo absoluto colisionaría con el resto de los segmentos (código, datos y *heap*), pero técnicamente puede expandirse de forma dinámica a lo largo de esos **2 GB**.

1. Diferencias entre una implementación iterativa y una recursiva (Memoria y Registros)

- **Enfoque Recursivo:** Es muy pesado para la computadora. Gasta muchísima memoria RAM porque cada vez que la función se llama, tiene que reservar espacio en la pila para guardar el `$ra` y el `$a0`. Además, es más lento porque ir a guardar y buscar datos en la memoria RAM toma bastante tiempo.
- **Enfoque Iterativo:** Es súper ahorrativo y rápido. Todo el proceso ocurre usando únicamente dos o tres registros internos del procesador (como `$t0` o `$v0`). La memoria RAM (la pila) ni siquiera se toca, por lo que su consumo de memoria es prácticamente cero.

2. Diferencias entre los libros y un ejercicio real en MIPS32

En los libros, los ejemplos suelen ser fragmentos de código "perfectos" que van directo a la lógica matemática. Asumen que por arte de magia los datos ya están cargados en los registros listos para usarse.

En la vida real, armar un código operativo en un simulador exige hacer todo el "trabajo sucio". Tienes que estructurar el archivo con sus secciones `.data` y `.text`, usar llamadas al sistema (`syscalls`) para pedirle números al usuario por teclado, imprimir los resultados en pantalla y, lo más importante, cerrar el programa limpiamente al final para que el simulador no arroje errores de ejecución.

Omisión de llamadas al sistema (`Syscalls`): Los libros asumen que los datos ya están mágicamente cargados en los registros para resolver el algoritmo rápido. En la realidad, el 40 por ciento del código de un ejercicio real se va en configurar `syscalls` para pedir datos al teclado, validar la entrada del usuario y dar formato a la salida en pantalla.

Falta de manejo de desbordamientos (`Overflow`): Un ejercicio de libro resuelve una suma o multiplicación usando `add` o `mult` directamente. Un ejercicio real debe usar instrucciones que capturen excepciones (`add` genera trampas de hardware) o usar operaciones lógicas (`addu`) junto con código adicional para verificar manualmente si el resultado superó los 32 bits y corrompió los datos.

Estructuras de control idealizadas: Los libros resuelven bucles e instrucciones condicionales (`if/else`) lineales y cortos. En la realidad, optimizar los saltos para que el procesador no pierda tiempo prediciendo mal el camino (`branch misprediction`) hace que el orden de las etiquetas y las condiciones en el código real se vea invertido o *enredado*, en comparación con la teoría limpia del papel.

Tutorial paso a paso de recursividad en MIPS32

La pila y su papel en la recursividad

Para comprender la recursividad a nivel de hardware, debemos analizar qué le ocurre a la CPU cuando se ejecuta la instrucción `jal` (*Jump and Link*).

- **La pérdida de contexto (\$ra):** `jal` salta a la etiqueta de la función y guarda la dirección de la siguiente instrucción en el registro `$ra`. Si la función se vuelve a llamar a sí misma dentro de su propio cuerpo, un segundo `jal` sobrescribirá el contenido de `$ra` con la nueva dirección. Sin la pila, la dirección de retorno original hacia el programa principal se pierde para siempre, atrapando al procesador en un bucle infinito.
- **La destrucción de parámetros (\$a0):** Al pasarle un nuevo argumento a la subrutina (por ejemplo, $n - 1$), el registro `$a0` debe modificarse. Si no respaldamos el valor anterior de n , al regresar de la llamada no tendremos el dato original para completar las operaciones matemáticas pendientes.

Estructura fundamental de un algoritmo recursivo en MIPS32

Fase 1: Expansión o Descenso (Generación de Marcos)

Cada vez que se evalúa el problema y no se ha alcanzado la parada, el programa ejecuta el prólogo:

- Resta espacio a la pila (`addi $sp, $sp, -8`).
- Almacena el `$ra` actual y el argumento `$a0` en la RAM (`sw`).
- Modifica `$a0` con el nuevo subproblema ($n - 1$) y realiza el salto `jal`.
- **Estado del hardware:** La pila se llena de "fotografías" del estado de la función en cada nivel de profundidad.

Fase 2: Interrupción o Punto de Inflexión (Caso Base)

- Se cumple la condición condicional (por ejemplo, `beq $a0, $zero, caso_base`).
- No se reserva más espacio en la pila ni se ejecuta `jal`.
- La función carga un valor inicial en el registro de retorno (`$v0`) y ejecuta inmediatamente un `jr $ra` para iniciar el regreso.

Fase 3: Contracción o Retorno (Resolución y Desapilado)

Al regresar a la llamada anterior, la función ejecuta el epílogo:

- Restaura el `$a0` y el `$ra` de ese nivel específico mediante `lw`.
- Incrementa `$sp` (`addi $sp, $sp, 8`) para devolver la memoria al sistema.
- Opera el valor recuperado de `$a0` con el resultado parcial acumulado en `$v0`.
- Ejecuta `jr $ra` para saltar a la instancia superior.

Implementación de referencia en MIPS32 (Algoritmo de Paridad):

```

paridad:
    # --- CASO BASE ---
    beq $a0, $zero, caso_base    # Si n == 0, detiene la recursión

    # --- PRÓLOGO (Reserva y guardado en la Pila) ---
    addi $sp, $sp, -8            # Reservar 8 bytes en la RAM
    sw   $ra, 0($sp)             # Preservar dirección de retorno
    sw   $a0, 4($sp)             # Preservar el parámetro 'n' actual

    # --- LLAMADA RECURSIVA ---
    addi $a0, $a0, -1            # Nuevo argumento: n - 1
    jal  paridad                 # Llamada recursiva paridad(n - 1)

    # --- COMBINACIÓN Y OPERACIÓN ---
    li   $t0, 1
    sub  $v0, $t0, $v0           # Inversión de paridad (1 - v0)

    # --- EPÍLOGO (Restauración y liberación de Pila) ---
    lw   $ra, 0($sp)            # Restaurar $ra de esta instancia
    lw   $a0, 4($sp)            # Restaurar $a0 original
    addi $sp, $sp, 8            # Devolver 8 bytes a la pila
    jr   $ra                    # Retornar al nivel superior

caso_base:
    li   $v0, 0                 # Caso base: 0 es par
    jr   $ra                    # Retornar

```

3. Justificación del enfoque Iterativo vs Recursivo

En ensamblador MIPS, el enfoque iterativo es el ganador indiscutible a nivel práctico.

Aunque la recursividad puede parecer más .^o elegante.º fácil de leer a nivel matemático, a nivel de hardware es muy ineficiente. El enfoque iterativo es muy superior porque se ejecuta en una fracción del tiempo y aprovecha al máximo la velocidad de los registros sin gastar memoria. En ensamblador, la recursividad se justifica casi exclusivamente como un ejercicio académico para aprender a manejar la memoria y la pila, pero rara vez es la mejor opción para un programa real que necesite ser rápido.

Aunque la iteración parece tener una clara ventaja ante la recursividad, hay problemas computacionales, matemáticos, etcétera, que necesitan un recorrido no lineal y automático, ya que la estructura del problema lo requiere.

Un ejemplo puede ser las torres de hanoi, definitivamente la versión recursiva muestra una clara ventaja, es como se dijo antes, una solución elegante, directa e intuitiva si se tiene los conocimientos básicos de la pila. Esta diferencia de enfoques no se nota mucho en ejercicios como el de la paridad, puesto que es un ejercicio de entrada a la recursividad, en los siguientes temas se nota la verdadera ventaja de la recursividad y se justifica el enfoque recursivo ante el iterativo:

Recorrido de estructuras de datos no lineales (Árboles y Grafos): Cuando te enfrentas a estructuras como árboles binarios, el DOM de páginas web o carpetas de archivos anidadas de profundidad desconocida, la iteración se vuelve extremadamente torpe. Para hacer un recorrido iterativo de un árbol, tendrías que programar y gestionar manualmente una pila (*stack*) auxiliar para recordar el flujo de ejecución. La recursividad hace esto de forma nativa utilizando la pila de llamadas del propio sistema operativo, traducándose en un código notablemente más limpio, legible y natural.

Algoritmos de División y Conquista (*Divide and Conquer*): Problemas complejos que se pueden fragmentar en subtarear idénticas más pequeñas —como *Merge Sort* o *Quicksort*— encuentran en la recursividad su solución ideal. Intentar replicar la lógica de división y combinación de estos algoritmos mediante bucles iterativos requiere una ingeniería de índices y arreglos muy propensa a errores y difícil de depurar.

Problemas matemáticos: La claridad conceptual y la fidelidad a la notación matemática formal que otorga la recursividad reducen drásticamente los posibles fallos en el planteamiento paso a paso del algoritmo.

Conclusión de la sección:

- Para el manejo de datos planos, secuencias lineales o algoritmos donde la rapidez y el procesamiento de volúmenes masivos de datos son críticos, la resolución **iterativa** es la opción ideal debido a su eficiencia en memoria.
- Para el manejo de estructuras ramificadas, enfoques de divide y vencerás, o para mantener la legibilidad ante lógicas iterativas excesivamente complejas, la resolución **recursiva** será la mejor opción.

4. Análisis y Discusión de los Resultados

Al comparar ambas implementaciones, los resultados nos demuestran que programar en bajo nivel te quita la red de seguridad que tienen los lenguajes modernos. La versión recursiva nos obligó a gestionar manualmente la preservación de los datos, demostrando lo fácil que es perder información si no se controla bien la pila y los saltos.

Por otro lado, la versión iterativa confirmó que el procesador está diseñado para trabajar mejor mediante ciclos repetitivos y condicionales directos. En conclusión, dominar ambos enfoques es esencial: la iteración nos enseña a escribir código rápido y óptimo, mientras que la recursión nos da una comprensión profunda de cómo el procesador interactúa con la memoria física.

Conclusión

El desarrollo de esta práctica permitió contrastar la teoría del diseño de software con la realidad física de la arquitectura MIPS32. Programar en bajo nivel elimina la red de seguridad de los entornos modernos, demostrando que conceptos abstractos como la recursividad dependen directamente de operaciones minuciosas en la memoria RAM y del manejo riguroso del puntero de pila (`$sp`).

En términos de rendimiento puro para algoritmos lineales, la resolución iterativa es el enfoque ganador indiscutible. Al operar exclusivamente en los registros internos del procesador, elimina el sobre costo de tiempo y memoria asociado a las instrucciones `sw` y `lw`. Sin embargo, el análisis realizado demostró que la recursividad no es un mero ejercicio académico: es una herramienta indispensable para abordar problemas no lineales (como el recorrido de árboles y grafos o algoritmos de divide y vencerás), donde la complejidad de gestionar una pila manual en un enfoque iterativo superaría los beneficios de rendimiento.

En última instancia, el dominio de ambos enfoques proporciona una comprensión profunda del equilibrio entre la elegancia algorítmica y la eficiencia de hardware, habilidad esencial para el diseño de sistemas optimizados y el entendimiento real del comportamiento del procesador.